holokai-neuron-sdk

SDK for building **Holokai neurons** in Python — connect to a BigBrain gateway over HTTP+SSE and execute capability-typed tasks.

This is the Python sibling of `@holokai/neuron-sdk` (TypeScript) and speaks the same wire protocol byte-for-byte (`packages/neuron-protocol`).

Install

```
pip install holokai-neuron-sdk
```

Requires Python 3.10+.

> **Gateway compatibility (protocol 1.2.0; HOL-2926, HOL-4326).** Capabilities may now advertise a contract (`description` , `inputSchema` , `outputSchema`) on the register frame. Gateways older than HOL-2926 validate frames strictly and reject contract-carrying registrations with `INVALID_FRAME` — deploy the gateway before releasing neurons built on this SDK version that set those fields.

Quick start

```python
import asyncio
import logging

from holokai_neuron_sdk import Capability, HandlerContext, Neuron


async def echo(input, ctx: HandlerContext):
    await ctx.progress(percent=50, message="halfway")
    return {"echo": input}


async def main():
    neuron = Neuron(
        gateway_url="https://api.holokai.dev",
        neuron_id="my-neuron-1",
        auth=lambda: "<bearer token>",  # sync or async; called per request
        logger=logging.getLogger("neuron"),
    )

    neuron.register_handler(
        Capability(type="example/echo", scope="any", concurrency=4),
        echo,
        input_schema={"type": "object"},
        output_schema={
            "type": "object",
            "properties": {"echo": {}},
            "required": ["echo"],
        },
    )

    await neuron.start()
    try:
        await asyncio.Event().wait()  # run until cancelled
    finally:
        await neuron.stop(drain=True, timeout=30)


asyncio.run(main())
```

Authentication

The `auth` callback supplies the bearer token for every POST and SSE open. How you obtain tokens depends on where the neuron runs (full model: `docs/NEURON_AUTH_SPEC.md` in the bigbrain repo):

- **Desktop-embedded neurons** run under the signed-in user: exchange the user's Moku JWT for an `aud=bigbrain` token (RFC 8693) and return it from `auth`.
- **Headless / enterprise neurons** enroll once via Moku's device flow, then mint short-lived tokens from the stored client credential — no static long-lived JWT anywhere.

Enroll (one-time, headless neurons)

```
holokai-neuron-sdk enroll --moku-url https://moku.example \
  --name my-neuron --capabilities examples/http.fetch,examples/web.search
# → prints a verification URL + code; an org admin approves in Moku.
# → credential written to ~/.holokai/neuron-credentials.json (0600)
```

Also available as `python -m holokai_neuron_sdk enroll …`, or programmatically via `enroll(...)` from `holokai_neuron_sdk.auth`.

Steady state

```python
from holokai_neuron_sdk import Neuron
from holokai_neuron_sdk.auth import create_client_credentials_auth

neuron = Neuron(
    gateway_url="https://api.holokai.dev",
    neuron_id="my-neuron-1",
    auth=create_client_credentials_auth(),  # reads ~/.holokai/neuron-credentials.json
)
```

`create_client_credentials_auth` mints `aud=bigbrain` tokens via the OAuth2 `client_credentials` grant, caches them against the response's `expires_in` (proactive refresh, single-flight), and re-reads the credential file on `invalid_client` so an operator-rotated secret is picked up without a restart. If the credential was revoked in Moku it raises `CredentialRevokedError` — re-enroll to recover.

Concepts

Concept	What it is
Neuron	A process that registers itself with a BigBrain gateway and offers one or more capabilities.
Capability	A namespaced task type (`owner/package[/name][.variant]`) with a scope (`"any"` or `{onBehalfOf: userId}`) and a per-capability concurrency.
Lease	A single in-flight task delivery. Carries the input, schemas, and a deadline. The neuron acks (success), nacks (failure), or progresses.
Cancel	The gateway can revoke a lease mid-flight. The handler's cancellation `asyncio.Event` is set, the task is cancelled, and a `cancelled` nack is sent.
Heartbeat	The SDK posts a heartbeat every 10s with the current in-flight lease list. The gateway uses it to detect dead neurons and to renew lease TTLs for long-running tasks.

Wire protocol

- **Transport**: SSE (server → neuron) + HTTPS POST (neuron → server).
- **Endpoints**:

    - `GET /neuron/events?neuronId=…` — SSE stream
    - `POST /neuron/{register, update-capability, heartbeat, ack, nack, progress}`

- **Authentication**: `Authorization: Bearer <jwt>`. The `auth` callback is invoked on every POST and SSE open, and is given one chance to refresh on 401 before the SDK escalates to reconnect-with-backoff.
- **Schemas**: input/output JSON Schema arrives on the lease frame; the SDK validates with `jsonschema` and emits a `schema-mismatch` nack on failure.
- **Versioning**: `PROTOCOL_VERSION = "1.0.0"` (free-form on the wire; the gateway negotiates compatibility).

Handler contract

```
async def handler(input, ctx: HandlerContext):
    # ctx.task         — the full task payload
    # ctx.lease_id     — the lease the gateway holds for this delivery
    # ctx.cancelled    — asyncio.Event set on cancel / shutdown
    # ctx.log          — logger scoped to this lease
    # ctx.progress(...) — non-blocking progress update
    # ctx.fail(reason, code=...) — terminal nack (do not catch)
    return {...}
```

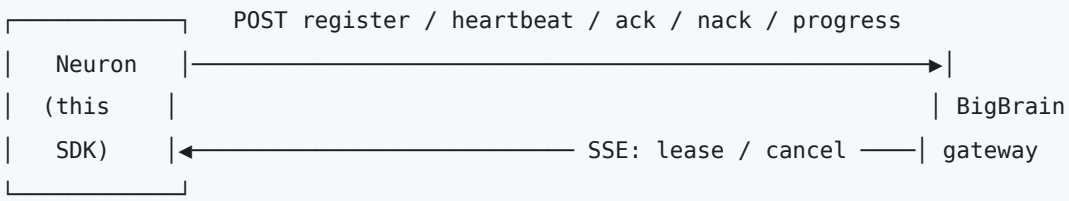
- **Sync handlers** are supported; the runner awaits awaitable return values and otherwise treats the return as the result.
- **Cancellation**: handlers wrapping I/O should cooperate with `ctx.cancelled` (e.g. `asyncio.wait` with `[ctx.cancelled.wait(), …]`).
- **Idempotency** is the handler author's responsibility — the framework does not checkpoint mid-task.

Failure classification

Outcome	Wire kind
Handler returns	`ack`
Handler raises `asyncio.CancelledError` (or cancel event set)	`nack: cancelled`
`ctx.fail(...)`	`nack: terminal`
Handler raises any other exception	`nack: retryable`
Input/output fails JSON Schema	`nack: schema-mismatch`

Retry policy lives on the workflow (server-side), never on the SDK.

Architecture



- `HttpTransport` owns one SSE stream and a shared `httpx.AsyncClient`.
- `Neuron` wires the transport to a heartbeat loop and a per-capability semaphore-bounded dispatcher.
- `run_lease(...)` validates input, invokes the handler, validates output, and posts the right ack/nack frame.

Examples

Example	Capability	What it does
`examples/http_fetch/`	`examples/http.fetch`	Fetches an arbitrary HTTP(S) URL via `httpx` and returns status/headers/body.
`examples/web_search/`	`examples/web.search`	Keyless web search via DuckDuckGo HTML scrape. No API key, but inherently brittle — see the example README for caveats.

Each ships with input/output JSON Schemas, cooperative cancellation (`ctx.cancelled`), and a CLI runner. Run them the same way:

```
pip install -e .
BIGBRAIN_GATEWAY_URL=https://api.holokai.dev \
BIGBRAIN_NEURON_ID=my-python-neuron-1 \
BIGBRAIN_TOKEN=eyJ... \
python -m examples.http_fetch    # or examples.web_search
```

Custom transports

`HttpTransport` is the default but the public `Transport` shape is documented on `TransportCallbacks` and `Neuron(transport=…)` accepts any object that matches the protocol — handy for tests or in-process embeddings.

Tests

```
# Unit + deterministic e2e (default — no network).
pytest

# Network-gated e2e: hits real DuckDuckGo through the real SDK to catch
# scraper drift. Skipped unless explicitly enabled.
E2E_NETWORK=1 pytest -m e2e_network
```

The e2e tests stand up a tiny aiohttp server on loopback that serves both the BigBrain `/neuron/*` surface and a stub for DuckDuckGo, then run the real `Neuron` SDK against it. Every byte flows over real HTTP — SSE in, POSTs out — exercising the full `register → lease → handler → ack` loop. See `tests/e2e/`.

License

MIT — see LICENSE.